Actor-Critic Model Predictive Control: Differentiable Optimization meets Reinforcement Learning

Angel Romero, Elie Aljalbout, Yunlong Song, Davide Scaramuzza

***Abstract*— An open research question in robotics is how to combine the benefits of model-free reinforcement learning (RL)—known for its strong task performance and flexibility in optimizing general reward formulations—with the robustness and online replanning capabilities of model predictive control (MPC). This paper provides an answer by introducing a new framework called *Actor-Critic Model Predictive Control*. The key idea is to embed a differentiable MPC within an actor-critic RL framework. This integration allows for short-term predictive optimization of control actions through MPC, while leveraging RL for end-to-end learning and exploration over longer horizons. Through various ablation studies, we expose the benefits of the proposed approach: it achieves better out-of-distribution behaviour, better robustness to changes in the dynamics and improved sample efficiency. Additionally, we conduct an empirical analysis that reveals a relationship between the critic's learned value function and the cost function of the differentiable MPC, providing a deeper understanding of the interplay between the critic's value and the MPC cost functions. Finally, we validate our method in a drone racing task on different tracks, in both simulation and the real world. Our method achieves the same superhuman performance as state-of-the-art model-free RL, showcasing speeds of up to 21 m/s. We show that the proposed architecture can achieve real-time control performance, learn complex behaviors via trial and error, and retain the predictive properties of the MPC to better handle out-of-distribution behavior.**

I. Introduction

The animal brain's exceptional ability to quickly learn and adjust to complex behaviors is one of its most remarkable traits, which remains unattained by robotic systems. This has often been attributed to the brain's ability to make both immediate and long-term predictions about the consequences of its actions and plan accordingly [1]–[3]. In robotics and control theory, model-based control has demonstrated a wide array of tasks with commendable reliability [4], [5]. In particular, Model Predictive Control (MPC) has achieved notable success across various domains such as the operation of industrial chemical plants [6], control of legged robots [7], and agile flight with drones [8]–[11]. The effectiveness of MPC stems from its innate capability for online replanning. This enables it to make decisions that optimize a system's future states over a specified short time horizon.

However, as tasks grow in complexity, model-based approaches necessitate substantial manual engineering, tailored to each specific task. This includes the careful crafting of the

The authors are with the Robotics and Perception Group, University of Zurich, Switzerland (http://rpg.ifi.uzh.ch). This work was supported by the European Union's Horizon 2020 Research and Innovation Programme under grant agreement No. 871479 (AERIAL-CORE) and the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT).

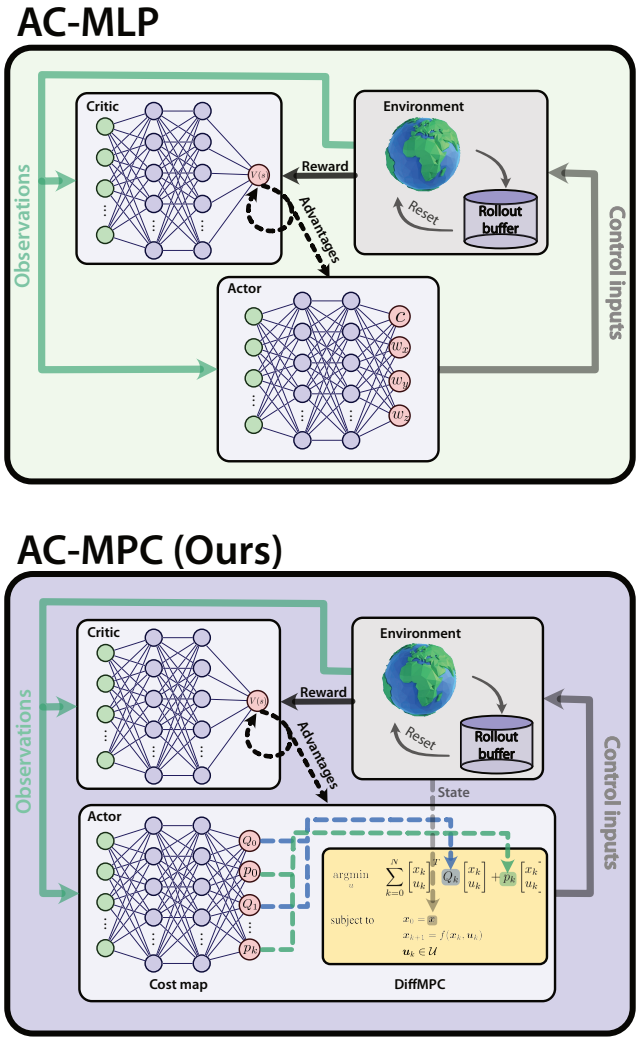


Fig. 1: **Top:** A block diagram of an actor-critic reinforcement learning architecture with a Multilayer Perceptron (MLP) **Bottom**: A block diagram of the proposed approach. We combine the strength of actor-critic RL and the robustness of MPC by placing a differentiable MPC as the last module of the actor policy. At deployment time, the commands for the environment are drawn from solving an MPC, which leverages the system's dynamics and finds the optimal solution given the current state. We show that the proposed approach achieves better out-of-distribution behavior and better robustness to changes in the dynamics. We also show that the predictions of the differentiable MPC can be used to improve the learning of the value function and the sample efficiency.

cost function, tuning of hyperparameters, and design of an appropriate planning strategy [10], [12]. Often, conservative assumptions about the task are made, leading to potentially sub-optimal task performance, for instance, in tasks where the dynamical system is taken to its limits [9], [10], or in applications that require discrete mode-switching [13]. Furthermore, the modular structure of model-based approaches may result in the progressive build-up of errors, accumulating in a cascading manner. This can compound inaccuracies, reinforce conservative estimations, and diminish the overall effectiveness of the system [14]–[16].

Most recently, reinforcement-learning-based control has gained considerable traction, demonstrating human-champion level or even superhuman performance in various domains, such as board games [17], atari games [18], complex video games [19], [20], and, in the physical world, in drone racing, both state-based [14] and vision-based [21]. In particular, the results in [14] demonstrate that model-free RL can achieve superior performance compared to optimal control techniques when pushing the limits in autonomous racing. This success is mainly due to the ability of RL to directly optimize a non-differentiable objective, eliminating the need for proxy objectives in the form of a predefined reference time trajectory or continuous 3D path. More generally, RL offers a flexible approach to control by directly optimizing a feedback policy through interactions with the environment, eliminating the need for intermediate representations. Unlike trajectory optimization or standard optimal control methods, RL can handle sparse reward signals (such as minimizing crashes, energy consumption, or lap times) and can improve robustness by leveraging domain randomization across diverse simulated conditions. This combination of offline learning, real-time adaptability, and robustness against uncertainty often makes RL more scalable and effective in complex scenarios where optimal control algorithms may struggle to yield real-time optimal solutions [14].

However, RL architectures are not without their own set of challenges [15], [22]. Learning a model-free, end-to-end policy without explicitly leveraging prior knowledge in the training process, such as physics or dynamic models, results in the need to learn everything from data, often resulting in millions of interactions within a simulator. While the end-to-end paradigm is attractive, it often lacks generalizability and robustness to out-of-distribution scenarios. This has resulted in hesitancy to apply end-to-end learned architectures to safety-critical applications and has fostered the development of approaches that advocate for the introduction of safety in learned pipelines [22]–[24].

This article is an extension of [25], which introduced an architecture called Actor-Critic Model Predictive Control to bridge the gap between Reinforcement Learning and Model Predictive Control. This architecture equips the agent with a differentiable MPC [26], placed after the last layer of the actor network, as shown in Fig. 1. This differentiable MPC module provides the system with online replanning capabilities and allows the policy to predict and optimize the short-term consequences of its actions.

Instead of relying on intermediate representations such as trajectories, we directly learn a map from observations to the cost function. Therefore, at deployment time, the control commands are drawn from solving an MPC, which leverages the system's dynamics and finds the optimal solution given the current state. The differentiable MPC module, which incorporates a model of the system's dynamics, provides the agent with prior knowledge even before any training data is received. The second component of our actor is the cost map, a deep neural network that encapsulates the dependencies between observations and the cost function of the MPC. In other words, while the differentiable MPC captures temporal variations inside its horizon, the neural cost module encodes the dependencies in relation to the observations. This architecture thereby incorporates two different time horizon scales: the MPC drives the short-term actions while the critic network manages the long-term ones. This represents a significant advantage over vanilla actor-critic RL, where the actor is typically a randomly initialized feedforward neural network with no domain-specific structure nor priors.

[25] demonstrated that the AC-MPC architecture exhibits more stable behavior in out-of-distribution scenarios, highlighting its robustness and generalizability compared to its neural-network-only counterpart. Lastly, [25] showcased the applicability of the proposed approach in real-world scenarios using an agile quadrotor platform, demonstrating the feasibility and effectiveness of the AC-MPC architecture in dynamic and challenging environments.

This paper extends [25] as follows:

- It improves the AC-MPC algorithm by incorporating predictions into the critic's training process. This improvement leverages the prior knowledge of the system through the MPC predictions and incorporates them in the learning process of the value function, resulting in improved sample efficiency during training.
- It conducts an empirical analysis that reveals a relationship between the Hessian of the learned value function and the learned cost functions of the differentiable MPC. This finding provides a deeper understanding of the interplay between the RL value and the MPC cost functions within the AC-MPC framework, offering valuable insights for future research.
- It shows that after optimizing hyperparameters, specifically exploration, AC-MPC can leverage the prior knowledge included in the dynamics of the system and achieve better training performance than AC-MLP.
- It shows that AC-MPC is more robust to dynamic parameter changes than the AC-MLP baseline, keeping high success rates for variations in mass, in XY inertia, and body rate limits. This improved robustness is attributed to the differentiable MPC's access to the dynamics model.
- It demonstrates in both simulation and real-world experiments that the proposed AC-MPC achieves superhuman performance on par with state-of-the-art model-free reinforcement learning in a drone racing task, attaining

speeds of up to 21 m/s on a quadrotor platform.

Overall, the AC-MPC architecture not only advances the integration of RL and MPC but also offers practical improvements in out-of-distribution behavior, training efficiency, and real-world applicability, contributing to the development of more robust and efficient control systems.

II. Related Work

Model Predictive Control (MPC) in robotics. MPC has had an enormous success in robotic applications over the last decades [9]–[11], [27]–[44]. However, often the cost function needs to be crafted by hand by an expert, and the hyperparameters need to be tuned at deployment time [45], [46]. Additionally, in most MPC setups [7], [9], [10], [47]–[52], the high-level task is first converted into a reference trajectory (planning) and then tracked by a controller (control). This decomposition of the problem into these distinct layers is greatly favored by the MPC methodology, largely due to the differentiability and continuity requirement of optimal control's cost functions. The cost function is shaped to achieve accurate trajectory tracking and is decoupled from (and usually unrelated to) the high-level task objective. As a result, the hierarchical separation of the information between two components leads to systems that can become erratic in the presence of unmodeled dynamics. In practice, a series of conservative assumptions or approximations are required to counteract model mismatches and maintain controllability, resulting in systems that are no longer optimal [14]–[16].

Another promising model-based direction for addressing complex control challenges is sampling-based MPC algorithms [53]–[56], which are designed to handle intricate, non-differentiable cost functions and general nonlinear dynamics. These algorithms integrate system dynamics – either known or learned – into the Model Predictive Path Integral (MPPI) control framework, optimizing control in real-time. A key feature of sampling-based MPC is the ability to generate a large number of control samples on-the-fly, often leveraging parallel computation via Graphics Processing Units (GPUs). However, implementing sampling-based MPC on embedded systems poses significant challenges, as it tends to be both computationally demanding and memory-intensive.

Reinforcement Learning (RL) in robotics. RL has risen as an attractive alternative to conventional controller design, achieving impressive performance that goes beyond model-based control in a variety of domains [17], [19], [57]–[60]. RL optimizes a controller using sampled data and can handle nonlinear, nonconvex, non-differentiable and even sparse objectives, manifesting great flexibility in controller design. Compared to model-based optimal control, RL has a number of key advantages: most importantly, RL can optimize the performance objective of the task directly, removing the need for explicit intermediate representations, such as trajectories. Moreover, RL can solve complex tasks from raw, high-dimensional sensory input, without the need of a specific metric state [14]. However, RL is still far from reaching the level of robustness and generalisability of model-based control approaches when deployed in the real world, due to its brittleness when deployed in situations outside of the training distribution, and its lack of guarantees [22]. Model-free methods generally use black-box optimization methods and do not exploit the first-order gradient through the dynamics, thus cannot leverage the full advantage of the prior knowledge.

Combinations of MPC and learning. Several methods have been developed to learn cost functions for MPC [61]–[75], dynamics models for MPC [38], [76]–[80], or both simultaneously [26], [40], [81]–[87]. For example, in [62], [63], a policy search strategy is adopted that allows for learning the hyperparameters of a loss function for complex agile flight tasks. On the other hand, the works in [38], [61] use Bayesian Optimization to tune the hyperparameters and dynamics of MPC controllers for different tasks such as car racing. Similarly, in [70] the authors propose a new method called *Deep Model Predictive Optimization* (DMPO) that learns the update rule of an MPPI controller using RL. They evaluate their algorihtm on a real quadrotor platform and outperform state-of-the-art MPPI algorithms.

Recent model-based RL approaches integrate MPC-like components either during policy learning [88], [89] or by using explicit MPC-based policies [81], [90]–[92]. Many of these methods learn a dynamics model and use it to perform MPC at inference time. The latter step is either done via random shooting [90], the cross-entropy method [91], [93], or a combination of these methods with a learned critic [81], [94].

Alternatively, approaches leveraging differentiability through optimizers have been on the rise. For example, for tuning linear controllers by getting the analytic gradients [95], for differentiating through an optimization problem for planning the trajectory for a legged robot [96], or for creating a differentiable prediction, planning and controller pipeline for autonomous vehicles [97], or for moving horizon estimation [98].

On this same direction, MPC with differentiable optimization [26], [99]–[101] proposed to learn the cost or dynamics of a controller end-to-end. In particular, the authors in [26] were able to recover the tuning hyperparameters of an MPC via imitation learning for non-linear, low-dimensional dynamics – such as cartpole and inverted pendulum – by backpropagating through the MPC itself. Later, in [102], the authors augment the cost function of a nominal MPC with a learned cost that uses the gradient through the optimizer for the task of navigating around humans. To train this learned cost, they also learn from demonstrations. Until [25], all these approaches were only demonstrated in the context of imitation learning. While imitation learning is effective, its heavy reliance on expert demonstrations is a burden. This dependence prevents exploration, potentially inhibiting its broader capabilities.

In [25], we address this issue by leveraging the advantages of both differentiable MPC and model-free reinforcement learning. By equipping the actor with a differentiable MPC, our approach provides the agent with online replanning capabilities and with prior knowledge, which is a significant

advantage over model-free RL, where the actor is a randomly initialized feedforward neural network. Unlike conventional MPC, our approach emphasizes robustness and adaptability, flexibly allowing for the optimization of intricate objectives through iterative exploration and refinement.

III. Methodology

In this section, we first present preliminaries about both Optimal Control and Reinforcement Learning, and then introduce our method.

A. Preliminaries

Consider the discrete-time dynamic system with continuous state and input spaces, $\boldsymbol{x}_k \in \mathcal{X}$ and $\boldsymbol{u}_k \in \mathcal{U}$ respectively. Let us denote the time discretized evolution of the system $f : \mathcal{X} \times \mathcal{U} \mapsto \mathcal{X}$ such that $\boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k)$, where the sub-index k is used to denote states and inputs at time t_k. The general Optimal Control Problem considers the task of finding a control policy $\pi(\boldsymbol{x})$, a map from the current state to the optimal input, $\pi : \mathcal{X} \mapsto \mathcal{U}$, such that the cost function $J : \mathcal{X} \mapsto \mathbb{R}^+$ is minimized,

$$\begin{aligned} \pi(\boldsymbol{x}) = \underset{\boldsymbol{u}}{\operatorname{argmin}} \quad & J(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{x}_0 = \boldsymbol{x}, \quad \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k) \\ & \boldsymbol{u}_k \in \mathcal{U}, \end{aligned} \tag{1}$$

where k ranges from 0 to N for x_k and from 0 to $N-1$ for $\boldsymbol{u}_k$. In the following sections, we will go through different ways in which $J(\boldsymbol{x})$ can be chosen.

B. Tracking Model Predictive Control

One of the primary uses of MPC is to control a dynamical system's behavior such that it closely follows a pre-computed reference trajectory. This is known as trajectory tracking, and the corresponding MPC formulation is referred to as *tracking MPC*. In tracking MPC approaches [103]–[105], the objective $J(\boldsymbol{x})$ is to minimize a quadratic penalty on the error between the predicted states and inputs, and a given dynamically feasible reference $\boldsymbol{x}_{k,ref}$ and $\boldsymbol{u}_{k,ref}$. Consequently, the cost function $J(\boldsymbol{x})$ in problem (1) is substituted by:

$$J_{MPC}(\boldsymbol{x}) = \sum_{k=0}^{N-1} \|\Delta \boldsymbol{x}_k\|^2_{\boldsymbol{Q}} + \|\Delta \boldsymbol{u}_k\|^2_{\boldsymbol{R}} + \|\Delta \boldsymbol{x}_N\|^2_{\boldsymbol{P}}, \tag{2}$$

where $\Delta \boldsymbol{x}_k = \boldsymbol{x}_k - \boldsymbol{x}_{k,ref}$, $\Delta \boldsymbol{u}_k = \boldsymbol{u}_k - \boldsymbol{u}_{k,ref}$, and where $\boldsymbol{Q} \succeq 0$, $\boldsymbol{R} \succ 0$ and $\boldsymbol{P} \succeq 0$ are the state, input, and final state weighting matrices. The norms of the form $\|\cdot\|^2_A$ represent the weighted Euclidean inner product $\|\boldsymbol{v}\|^2_A = \boldsymbol{v}^T A \boldsymbol{v}$.

This formulation relies on the fact that a feasible reference $\boldsymbol{x}_{k,ref}$, $\boldsymbol{u}_{k,ref}$ is accessible for N time steps in the horizon. Searching for this reference trajectory is often referred to as *planning*, which, depending on the application, can be both computationally intensive and complex, particularly in scenarios involving complex dynamics or cluttered environments [9], [106], [107]. In many cases, solving the planning problem becomes the bottleneck in real-time applications due to high computational requirements and the need for precise system models.

The tracking MPC formulation also relies on the assumption that a trajectory represents a well-posed, quadratic proxy objective for solving the end task. While this assumption holds for many practical use cases, it becomes limiting in situations where the system operates near its performance limits, such as aggressive maneuvers or high-speed tasks, where the trajectory may no longer serve as an effective proxy. In such cases, the tracking objective can result in suboptimal performance or even task failure [14].

C. Economic Model Predictive Control

To overcome these limitations, Economic MPC directly incorporates the task objective into the optimization problem, eliminating the need for a pre-defined reference trajectory. This fundamental difference allows for a more flexible cost function design, enabling the controller to be tailored to the specific economic objectives of the task [108], [109]:

$$J_{EMPC}(\boldsymbol{x}) = \sum_{k=0}^{N-1} l(x_k, u_k), \tag{3}$$

where $l(x_k, u_k)$ is directly the stage cost. This stage cost is generally designed to optimize sparse metrics, such as energy consumption, efficiency, or success rate.

While economic MPC offers advantages in terms of flexibility with respect to task design, it also presents several challenges that must be addressed for successful implementation. One of the primary caveats is that the stage cost $l(x_k, u_k)$ in most tasks tends to be sparse and non-differentiable, which complicates the formulation of Eq. (3). For example, in tasks with discontinuous behavior (e.g., switching between different modes, dealing with contact forces, or dealing with agent crashes), it is generally difficult to find a cost function that is differentiable.

D. General Quadratic MPC formulation

In most MPC approaches, there is a need of i) an explicit manual selection of a differentiable cost function that properly encodes the end task, and ii) hyperparameter tuning. As mentioned in section III-B, in the case of a standard tracking MPC this encoding is done through planning, by finding a dynamically feasible reference trajectory that translates the task into suitable cost function coefficients for every time step. For economic MPC (section III-C), recent research has been going in the direction of using data to approximate the cost function [110], or making it computationally tractable by transforming it to a quadratic program using the Hessian [111], [112].

However, these approaches present two main drawbacks: i) hand-crafting a dense, differentiable cost function can be difficult for a general task, and ii) even if this cost function is found, extra effort needs to be spent in fine tuning the hyperparameters for real-world deployment. More generally, optimization-based architectures such as MPC need to run in real-time when deployment in the real world is desired. As

a result, the underlying optimization problem is often simplified by approximating the original nonlinear formulation and converting it into a Quadratic Program (QP) suitable for the real-time iteration (RTI) [113]–[115] scheme to ensure computational tractability. A general quadratic cost function can be written as

$$J_{QP}(\boldsymbol{x}) = \sum_{k=0}^{N} \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T Q_k \begin{bmatrix} x_k \\ u_k \end{bmatrix} + p_k \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \quad (4)$$

In this paper, we propose to directly search for the matrix coefficients of Eq. (4) (i.e., Q_k and p_k) using reinforcement learning and differentiable MPC [26]. This way, we can encode the task as a general, non-differentiable reward function while at the same time maintaining computational tractability.

E. Actor-Critic Reinforcement Learning

Reinforcement learning problems are often framed within the Markov Decision Process (MDP) formalism, which provides a structured approach for modeling decision-making problems. An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the set of possible states, $\mathcal{A}$ is the set of possible actions, $P(s_{k+1}|s_k, a_k)$ is the state transition probability, $R(s_k, a_k)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The objective in an MDP is to find a policy π that maximizes the expected cumulative reward over time. Within the context of Problem (1), and when the state transition function is deterministic, it is equivalent to the system dynamics $f(x_k, u_k)$, where in this case the observation s_k is directly the state x_k and the action a_k is directly the control input u_k.

To solve such problems, a common approach in Reinforcement Learning is to optimize directly the policy π_θ by using the policy gradient. The policy π_θ is parameterized by θ, typically the weights of a neural network. The goal is to adjust these parameters to maximize the expected return, defined in the infinite-horizon case as:

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r(s_k, a_k), \quad (5)$$

where τ denotes a trajectory, $r(s_k, a_k)$ the reward function, and $\gamma \in [0, 1)$ is a discount factor that discounts future rewards. Unlike real-time optimization in methods like MPC, RL can directly optimize a non-differentiable, sparse reward. Additionally, policy gradient optimization is typically performed offline through interactions with simulation. Once trained, the policy π_θ enables the computation of control signals by simply evaluating the function $a^* = \pi_\theta(s_k)$ at each time step, reducing the computational complexity during deployment. For Actor-Critic reinforcement learning, the key idea is to simultaneously learn a state-value function $V_\omega(s)$ and learn a policy function π_θ, where the value function (Critic) and policy (Actor) are parameterized by ω and θ separately. The policy is updated via the policy gradient theorem [116],

$$\nabla_\theta J(\pi_\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{T} \nabla_\theta \log \pi_\theta(a_k^i | s_k^i) A_\omega(s_k^i, a_k^i), \quad (6)$$

where $A(s_k, a_k) = r(s_k, a_k) + \gamma V_\omega(s_{k+1}) - V_\omega(s_k)$ is the advantage function used as a baseline. In a standard actor-critic method, the policy is a stochastic representation where $a_k = \pi_\theta(s_k)$ is the mean of a Gaussian distribution.

F. Actor-Critic Model Predictive Control

This paper proposes an Actor-Critic MPC controller architecture where the MPC is differentiable [26] and the cost function is learned end-to-end using RL. The MPC block is introduced as the differentiable module of the actor in an actor-critic pipeline that uses the proximal policy optimization (PPO) algorithm [117]. Our architecture is shown in Fig. 1. In contrast to previous work [62], [118], where the MPC is taken as a black-box controller and the gradient is sampled, in our case the gradient of the cost function with respect to the solution is analytically computed and propagated using a differentiable MPC [26]. Therefore, for every backward and forward pass of the actor network, we need to solve an optimization problem. Instead of resorting to task-specific engineering of the cost function, we propose a neural cost map where the Q_k and p_k terms are the output of the neural network layers preceding the differentiable MPC block. This allows us to encode the end task directly as a reward function, which is then trainable end-to-end using the PPO training scheme. The main benefit of this approach with respect to training a standard Multi-Layer Perceptron (MLP) end-to-end is that the final module of the actor is a model-based MPC controller,

$$u_k \sim \mathcal{N}\{\text{diffMPC}(x_k, Q(s_k), p(s_k)), \Sigma\}, \quad (7)$$

and therefore it retains its generalizability and robustness properties. The model-based controller placed after the last layer of the actor network ensures that the commands are always feasible for the dynamics at hand, and that they respect the system constraints. This differentiable MPC supports control input constraints but not state constraints, which is why we add $\boldsymbol{u}_k \in \mathcal{U}$ in (1). To allow for exploration, during training the control inputs are sampled from a Gaussian distribution where the mean is the output of the MPC block, and the variance is controlled by the PPO algorithm. However, during deployment the output from the MPC is used directly on the system without further sampling, retaining all properties of a model-based controller.

G. Neural Cost Map

The cost function for the model predictive control architecture presented in Section III-F is learned as a neural network, depicted in Fig. 1 as *Cost Map*. Several adaptations to the system are needed in order to properly interface the neural network architecture with the optimization problem. First,

Algorithm 1: Actor-Critic Model Predictive Control

Input: initial neural cost map, initial value function V
for $i = 0, 1, 2, \cdots$ **do**
  Collect set of trajectories $\mathcal{D}_i\{\tau\}$ with $u_k \sim \mathcal{N}\{\text{diffMPC}(x_k, Q(s_k), p(s_k)), \Sigma\}$
  Compute reward-to-go $\hat{R}_k$
  Compute advantage estimates $\hat{A}_k$ based on value function $V(s_k)$
  Update the cost map by policy gradient (e.g., PPO-clip objective) and diffMPC backward [26]
  Fit value function by regression on mean-squared error
Output: Learned cost map

we construct the matrix $Q(s_k)$ and vector $p(s_k)$ as follows:

$$Q(s_k) = \text{diag}(Q(s_k)_{x_1}, \dots, R(s_k)_{u_1}, \dots)$$
$$p(s_k) = [p(s_k)_{x_1}, \dots, p(s_k)_{u_1}, \dots] \qquad \forall k \in 0, \dots, T,$$

where $x_1, \dots$ and $u_1, \dots$ are the states and inputs to the system, respectively, and $Q(s_k)$ and $p(s_k)$ are the learnable parameters, interface from the neural network to the optimization problem.

The purpose of the diagonalization of the Q matrix is to reduce the dimensionality of the learnable parameter space. Therefore, the dimensionality of the output dimension of the *Cost Map* is $2T(n_{state} + n_{input})$. In order to ensure the positive semi-definiteness of the Q matrix and the positive definiteness of the R matrix, a lower bound on the value of these coefficients needs to be set. To this end, the last layer of the neural cost map has been chosen to be a sigmoid which allows for upper and lower bounds on the output value. These lower and upper limits are chosen equal for Q and p, of 0.1 and 100000.0, respectively. Therefore, the final neural cost map consists of two hidden layers of width 512 with ReLUs in between and a sigmoid non-linearity at the end. The critic network consists also of two hidden layers of width 512 and ReLUs. The ouput of the critic network is a scalar.

H. Model-Predictive Value Expansion

At each control iteration, the differentiable MPC block in Fig. 1 outputs a sequence of optimal states and actions, of which only the first action u_0 is applied to the system. The remaining predicted states and actions, although currently discarded, contain valuable information. They can be used to improve the learning of the value function.

In this section, we propose an extension to our algorithm that makes use of the predictions of the MPC to improve the quality of the value function estimate. We call this extension Model-Predictive Value Expansion (MPVE). This idea is inspired by the Model-Based Value Expansion algorithm proposed by [119], which uses the extra predicted rollouts using a learned model to improve the value function estimate. Assuming that our model is accurate up to depth H and that we have access to the reward of the predictions, one can define the H-Step Model-Predictive Value Expansion as

$$\hat{V}_H(s) = \sum_{t=0}^{H-1} \gamma^t \hat{r} + \gamma^H \hat{V}(s_H), \tag{8}$$

where $\hat{r}$ represents the predicted rewards, and $\hat{V}(s_H)$ is the estimated value of the predicted state at the end of the horizon H.

However, as explained in [119], a challenge that arises is the distribution mismatch. This occurs when the distribution of states seen during training differs from the distribution of states encountered when using the predictions from the differentiable MPC. This mismatch can degrade performance, as highlighted in [119].

To address this issue, the work in [119] incorporates the TD k-trick, an approach designed to mitigate the distribution mismatch problem by aligning the training distribution with the prediction's distribution over multiple steps. The TD k-trick involves training the value function on k-step returns, which helps ensure the training and prediction distributions are closely aligned.

Therefore the expression for the value loss is extended

$$\frac{1}{H} \sum_{t=0}^{H-1} \left(V(\hat{s}_t) - \left(\sum_{k=t}^{H-1} \gamma^{k-t} \hat{r}_k + \gamma^H \hat{V}(\hat{s}_H) \right) \right)^2. \tag{9}$$

This additional term encourages the value function to be consistent with the MPC predictions across multiple time steps, thereby improving the overall accuracy of the value estimates.

We incorporate Eq. (9) into our original algorithm (Algorithm 1), to create a more robust learning process that effectively utilizes the predictive power of the MPC. This integration allows for better exploitation of the model's predictions, potentially leading to faster convergence and improved policy performance. After including Eq. (9) in algorithm 1, this is how the algorithm changes:

Algorithm 2: Actor-Critic Model Predictive Control with Model-Predictive Value Expansion

Input: initial neural cost map, initial value function V
for $i = 0, 1, 2, \cdots$ **do**
  Collect set of trajectories and predictions $\mathcal{D}_i\{\tau\}$ with $x_{k:k+H}, u_{k:k+H} \sim \mathcal{N}\{\text{diffMPC}(x_k, Q(s_k), p(s_k)), \Sigma\}$
  Compute reward-to-go $\hat{R}_k$ and value targets using TD(λ)
  Compute advantage estimates $\hat{A}_k$ based on value function $V(s_k)$
  Compute reward-to-go for predictions $\hat{R}_{k:k+H}$ and value targets using TD k-trick
  Update the cost map by policy gradient (e.g., PPO-clip objective) and diffMPC backward [26]
  Fit value function by regression on mean-squared error using a sum of the TD(λ) value loss and Eq. (9)
Output: Learned cost map

IV. Experimental Setup

This section presents a set of experiments, both in simulation and the real world. All experiments have been conducted using a quadrotor platform. To showcase the capabilities of our method, we have chosen the task of agile flight through a series of gates in different configurations: horizontal, vertical, circular, and SplitS. Additionally, to show the sim-to-real transfer capabilities, both circular and SplitS tracks are

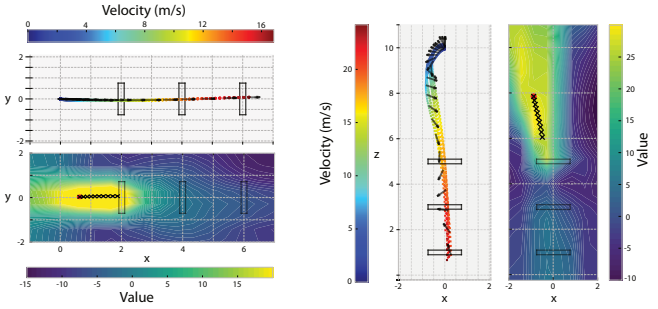


Fig. 2: **Actor-Critic Model Predictive Control (AC-MPC) applied to agile flight:** velocity profiles and corresponding value function plots. The left side illustrates horizontal flight, while the right side shows vertical flight. In the value function plots, areas with high values (depicted in yellow) indicate regions with the highest expected returns. The MPC predictions are shown as black Xs.

deployed in the real world. We train in a simple simulator in order to speed up the training and evaluate in BEM, a high-fidelity simulator [120], which has a higher level of similarity in terms of aerodynamics with the real world. The quadrotor platform's dynamics are the same as in [14]. For every different task, the policies are retrained from scratch. Lastly, all experiments have been conducted using a modification of the *Flightmare* software package [121] for the quadrotor environment and PPO implementation and *Agilicious* [122] for the simulation and deployment.

A. Observations, actions and rewards

1) Observation space: For all tasks presented in our manuscript, the observation space does not change, and it consists of two main parts: the vehicle observation $\mathbf{o}_t^{\text{quad}}$ and the race track observation $\mathbf{o}_t^{\text{track}}$. We define the vehicle observation as $\mathbf{o}_t^{\text{quad}} = [\boldsymbol{v}_t, \mathbf{R}_t] \in \mathbb{R}^{12}$, which corresponds to the quadrotor's linear velocity and rotation matrix. We define the track observation vector as $\mathbf{o}_t^{\text{track}} = [\delta\boldsymbol{p}_1, \cdots, \delta\boldsymbol{p}_i, \cdots]$, $i \in [1, \cdots, N]$, where $\delta\boldsymbol{p}_i \in \mathbb{R}^{12}$ denotes the relative position between the vehicle center and the four corners of the next target gate i or the relative difference in corner distance between two consecutive gates. Here $N \in \mathbb{Z}^+$ represents the total number of future gates. This formulation of the track observation allows us to incorporate an arbitrary number of future gates into the observation. We use $N = 2$, meaning that we observe the four corners of the next two target gates. We normalize the observation by calculating the mean and standard deviation of the input observations at each training iteration.

2) Action space: Previous work has demonstrated the high importance of the action space choice for learning and sim-to-real transfer of robot control policies [123], [124]. In this work, the control input modality is collective thrust and body rates, which was previously shown to perform best for agile flight [123]. This action is expressed as a 4-dimensional vector $\mathbf{a} = [c, \omega_x, \omega_y, \omega_z] \in \mathbb{R}^4$, representing mass-normalized thrust and body rates, in each axis separately. Even if the MPC block uses a model that limits the actuation at the single rotor thrust level, collective thrust and body rates are computed from these and applied to the system. This ensures that the computed inputs are feasible for the model of the platform.

3) Rewards: For all experiments, one reward term in common is the gate progress reward, which encourages fast flight through the track. The objective is to directly maximize progress toward the center of the next gate. Once the current gate is passed, the target gate switches to the next one. At each simulation time step k, the reward function is defined by

$$r(k) = \begin{cases} -10.0 & \text{if collision,} \\ +10.0 & \text{if gate passed,} \\ +10.0 & \text{if race finished,} \\ \|g_k - p_{k-1}\| - \|g_k - p_k\| - b\|\boldsymbol{\omega}_k\| & \text{otherwise.} \end{cases} \tag{10}$$

where g_k represents the target gate center, and p_k and p_{k-1} are the vehicle positions at the current and previous time steps, respectively. Here, $b\|\boldsymbol{\omega}_k\|$ is a penalty on the body rate multiplied by a coefficient $b = 0.01$.

V. Results

We perform multiple experiments to better understand different properties of our proposed approach. Namely, we study (i) AC-MPC's performance on multiple drone racing tasks and its sample efficiency in comparison to AC-MLP policies, (ii) its robustness to external disturbances, (iii) its robustness to changes in the dynamics, (iv) the benefits of the Model-Predictive Value expansion, (v) the interpretability properties of AC-MPC, (vi) its sensitivity to exploration hyperparameters, (vii) its real-world deployment. Each aspect corresponds to a different research question that we address in each of the following sections.

A. Does our method improve performance and sample efficiency?

We start with horizontal and vertical flight through gates. The vertical task can show if the approach is able to find a solution that lies directly in the singularity of the input space of the platform since the platform can only generate thrust in its positive body Z direction. When flying fast downwards, the fastest solution is to pitch and roll the drone as soon as possible, direct the thrust downwards, and only then command positive thrust [9]. However, many approaches are prone to get stuck in a local optimum [10], where the commanded thrust is zero and the platform gets pulled only by gravity. Fig. 2 shows the simulation results of deploying the proposed approach, which was trained in the horizontal and vertical tracks (left and right side of Fig. 2, respectively). We show velocity profiles and value-function profiles. The value-function profiles have been computed by selecting a state of the platform in the trajectory and modifying only the position while keeping the rest of the states fixed. For the horizontal track, we sweep only the XY positions, and for the vertical track, the XZ positions. Additionally, 10 MPC predictions are shown and marked with Xs. In these value function plots, areas with high values (in yellow) indicate regions with high expected returns.

In Fig. 2 and in the supplementary video, one can observe the evolution of the value function over time. Given the sparse nature of the reward terms (see Section IV-A.3), one can observe that when a gate is successfully passed, the region of high rewards quickly shifts to guide the drone towards the next gate. This can be interpreted as a form of discrete mode switching enabled by the neural network cost map. Such mode-switching behavior is a challenging feat to accomplish using traditional MPC pipelines. The intuition behind this is that the critic is able to learn long-term predictions, while the model-predictive controller focuses on the short-term ones, effectively incorporating two time scales.

Additionally, we train our approach on a challenging track known as the SplitS track. Designed by a professional drone racing pilot, this track is distinguished by its highly demanding SplitS maneuver, where the drone must navigate through two gates placed directly above one another successively. This track was used in previous research as a benchmark, namely [9]–[11], [14], [21].

In Fig. 3 we show the collective thrust commands of both approaches, AC-MLP vs AC-MPC. In this figure, one can notice how the AC-MPC can achieve saturation in a consistent way, while the AC-MLP approach also saturates but in a more irregular fashion. This indicates that the MPC module within the AC-MPC framework, which includes explicit knowledge of both platform dynamics and its constraints, effectively and consistently utilizes the system's saturation limits. In contrast, the AC-MLP commands are generated directly by a neural network, resulting in behavior that is comparatively less consistent and less aligned to the platform's true limits.

In Fig. 4, we can see the reward evolution for AC-MLP when compared with AC-MPC. This comparison was conducted after optimizing the initial standard deviation hyperparameter for both approaches and choosing the best result for each. Generally—as we also study in section V-F—AC-MPC performs better for lower exploration parameters, whereas AC-MLP needs higher exploration parameters. Fig. 4 shows that the asymptotic training performance and sample efficiency are improved when choosing AC-MPC, since it is able to leverage the prior information encoded in the dynamics within the differentiable MPC. This is particularly evident in the reward evolution observed for the Vertical and SplitS tracks. For simpler tasks, the prior knowledge introduced in AC-MPC is not strongly advantageous to performance. For instance, in the Horizontal track, which is relatively simple, the training performance of AC-MLP is on par with AC-MPC, since the maneuvers that the policy needs to discover to perform the task do not require an elaborate exploration behavior.

In conclusion, the results presented in this section show that our method improves both performance and sample efficiency. By comparing AC-MPC against AC-MLP across various track complexities, and after hyperparameter tuning, we have demonstrated that AC-MPC can achieve superior performance and sample efficiency, especially in challenging tasks like the Vertical and SplitS tracks (see Fig. 4). This improvement is attributed to AC-MPC's ability to effectively leverage prior knowledge of the system dynamics through the embedded MPC module (see Fig. 3). These findings suggest that incorporating model-based control elements into reinforcement learning frameworks offers a promising direction for future research.

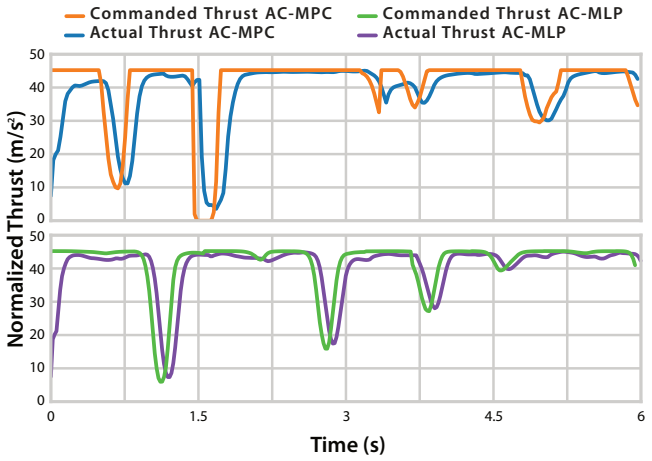


Fig. 3: Evolution of the collective thrust command over time for both AC-MLP and AC-MPC during trials on the SplitS track. The figure demonstrates AC-MPC's ability to effectively utilize control input saturation due to its access to the system dynamics and control constraints. In contrast, AC-MLP exhibits less consistent saturation behavior.

TABLE I: Comparison: Success Rate and average velocity for agile flight through the Horizontal, Vertical and Vertical with wind tracks.

	Horizontal		**Vertical**		**Vertical Wind**	
	SR [%]	v [m/s]	**SR** [%]	v [m/s]	**SR** [%]	v [m/s]
AC-MLP	74.78	7.74	53.61	10.56	6.5	10.67
MPC	64.94	4.15	**72.27**	4.25	0.0	6.44
AC-MPC	**90.37**	6.51	64.47	10.05	**83.33**	10.76

B. Is our approach robust to external disturbances?

We perform various studies where the standard actor-critic PPO architecture [14] (labeled as *AC-MLP*) and a standard tracking MPC are compared to our approach (labeled as *AC-MPC*) in terms of generalization and robustness to disturbances. AC-MLP and AC-MPC approaches are trained with the same conditions (reward, environment, observation, simulation, etc.). All these evaluations are conducted using the high-fidelity BEM simulator [120]. The MPC approach tracks a time-optimal trajectory obtained from [9]. This is because, when using AC-MPC, we impose a modular dynamic structure compared to the flexibility of the single neural network used by the AC-MLP architecture.

In terms of disturbance rejection and out-of-distribution behavior, we conduct three ablations, shown in Fig. 5 and Table I). In Fig. 5A (and the *Vertical Wind* column of Table I), we simulate a strong wind gust that applies a constant external force of 11.5 N (equivalent to 1.5x the weight of the platform). This force is applied from $z = 10m$ to $z = 8m$.

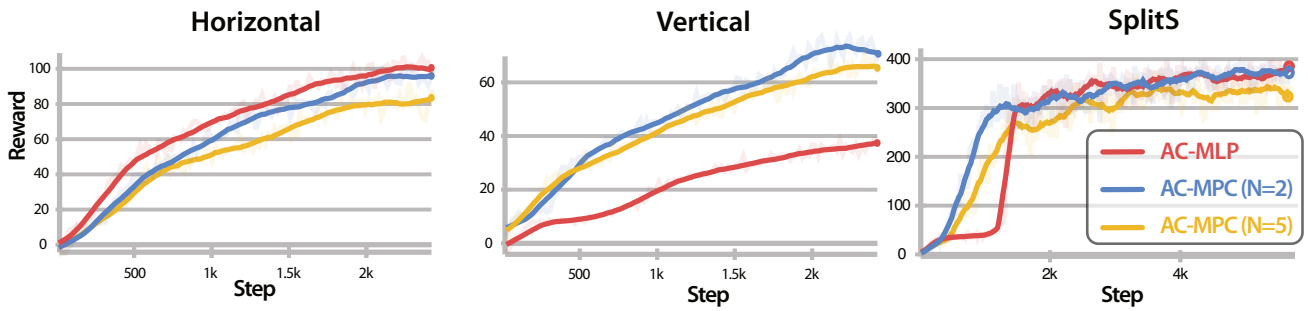


Fig. 4: Reward evolution for different tracks, for AC-MLP (for different N values of the horizon, $N = 2$ and $N = 5$) and AC-MPC. The values have been obtained after optimizing the initial exploration standard deviation for both approaches independently, and then selected the best result from each. For Vertical and SplitS tracks, one can observe how the AC-MPC approach is able to leverage its prior knowledge of the system and showcase improved learning. For the Horizontal track, because of its simplicity, the leverage of prior knowledge is diluted.

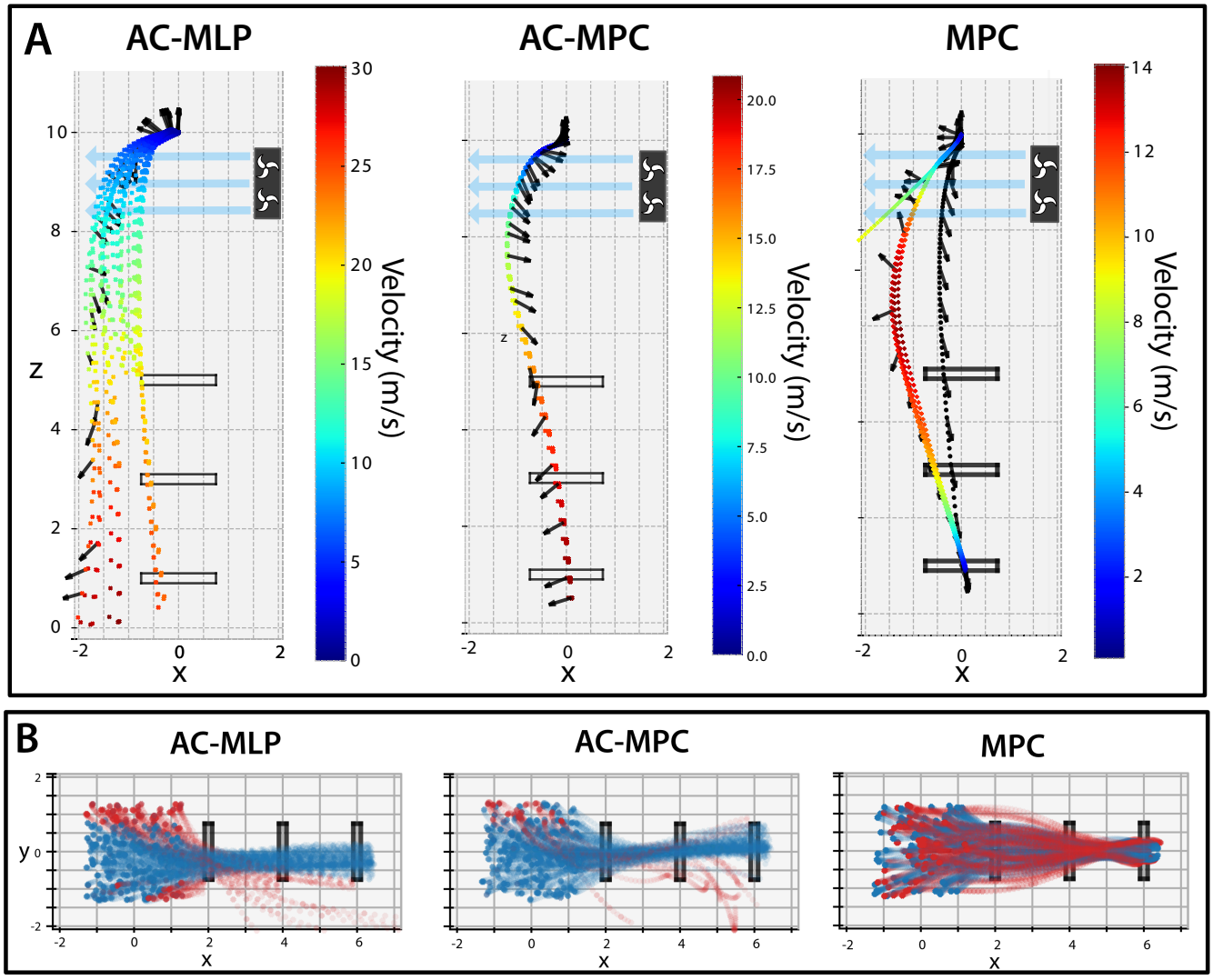


Fig. 5: **Baseline comparisons between our AC-MPC, a standard PPO (termed AC-MLP), and a standard tracking MPC. (A)**: Robustness against wind disturbances (vertical track). All policies are trained without disturbances. Black arrows indicate the quadrotor's attitude. **(B)**: Robustness against changes in initial conditions (horizontal track). Trajectories are color-coded, with crashed trajectories in red and successful in blue.

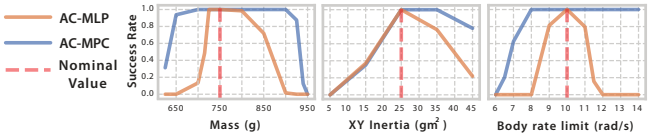


Fig. 6: Success rate against variations of mass, inertia, and body rate limit. Policies are trained in the SplitS track with a nominal value for these parameters. This nominal value is depicted in red. Then, these policies are deployed in 64 parallel environments, each environment having slightly different initial position. In the deployment stage, mass, inertia, and body rate limit are varied. The success rate is computed as the proportion of trajectories that don't have crashes. Because AC-MPC has the dynamics embedded in the network architecture, it shows to be more robust to these changes than its AC-MLP counterpart.

We can see how neither the AC-MLP nor the MPC policies can recover from the disturbance and complete the track successfully. On the other hand, AC-MPC achieves a higher success rate (83.33%, as shown in Table I), and exhibits more consistency among repetitions. This showcases that incorporating an MPC block enables the system to achieve robustness.

For the *Vertical* and *Horizontal* experiments in Table I, we simulate 10000 iterations for each controller where the starting points are uniformly sampled in a cube of 3m of side length where the nominal starting point is in the center. In the *Horizontal* case, the results are shown in Fig. 5B. It is important to highlight that during training of AC-MLP and AC-MPC, the initial position was only randomized in a cube of 1m of side length. The successful trajectories are shown in blue in Fig. 5B, while the crashed ones are shown in red. In Table I, we can observe that the AC-MPC presents a higher success rate than AC-MLP in both experiments. One can also see that AC-MPC has a higher success rate than the MPC approach in the *Horizontal task*, but this is not the case in the *Vertical* task. The reason behind this is that in the *Vertical* task, the MPC is not able to track the solution that turns the drone upside down, therefore resulting in the sub-optimal solution of setting all thrusts to near-zero state and dropping only by the effect of gravity, which results in slower but safer behavior. This is evident by looking at the average speed column.

These experiments provide empirical evidence showing that AC-MPC exhibits enhanced performance in handling unforeseen scenarios and facing unknown disturbances, which makes it less brittle and more robust.

C. Is our approach robust to changes in the dynamics?

Since our approach integrates a model-based controller that explicitly accounts for system dynamics, the question arises of whether AC-MPC is sensitive to dynamic parameters changes when compared to AC-MLP. To investigate this, we conduct a series of experiments in which key dynamic parameters are systematically varied. Both policies are trained on the SplitS track using nominal parameter values, after which they are evaluated across a range of perturbed parameters for both AC-MLP and AC-MPC. To quantify the robustness of these policies, we deploy them from 64 different positions, equally spread on a cube of $0.5\,m$ of side length. Success is defined as the percentage of trajectories that successfully navigate through all gates of the SplitS track. Fig. 6 presents the success rate as a function of variations in mass (up to $+27\%$), in XY inertia (up to $\pm 80\%$) , and body rate limits (up to $\pm 40\%$).

The results demonstrate that the AC-MPC exhibits superior generalization to dynamic parameter variations compared to the AC-MLP. This improved robustness is attributed to the incorporation of a dynamic model within the differentiable MPC block, which allows for adjustments in response to perturbed dynamics during deployment without retraining.

D. What do we gain from Model-Predictive Value Expansion?

In this section, we conduct empirical experiments to evaluate the effect of the AC-MPC extended with Model-Predictive Value Expansion algorithm (introduced in Section III-H) in terms of training performance. To isolate the effect of MPVE and enable clear reward evaluation, we employ a straightforward hovering task for the drone. Here, the drone starts at a random position and orientation within a 1x1x1 meter cube around a designated equilibrium point (0, 0, 5 meters). The objective is to stabilize the drone at this point with zero velocity and a perfectly upright orientation (hover). We compare the training performance of AC-MPC-MPVE against AC-MLP. Both algorithms are trained with a fixed sample budget of 50 steps, corresponding to roughly 2 million samples collected from the environment. Because the MPVE algorithm primarily affects the learning of the critic, which uses the Generalized Advantage Estimation (GAE) algorithm [125], we explore the impact of our extension for various values of the temporal difference parameter λ in TD(λ). The parameter λ determines the trade-off between bias and variance when estimating returns. A lower value of λ favors lower variance but higher bias by relying more on immediate rewards, while higher values reduce bias but increase variance by incorporating more long-term information. To investigate this, we experiment with six different values for λ: 0.0, 0.2, 0.4, 0.6, 0.8, and 1.0. For each value, we repeat the training process three times with different random seeds.

Our results (visualized in Fig. 7) demonstrate that the sample efficiency gains achieved by MPVE are more significant for lower lambda values. This can be explained by the way the value function leverages data from the rollout buffer. Higher lambda values incorporate data from a larger portion of the rollout, increasing the influence of Monte Carlo estimates and reducing the relative impact of the information provided by MPVE predictions. In contrast, lower lambda values lead to stronger benefits from MPVE. This suggests that our algorithm is particularly advantageous for scenarios where the value function is trained with TD(0). This approach is often preferred when memory limitations exist (as it avoids storing the entire rollout buffer) or for non-episodic tasks.

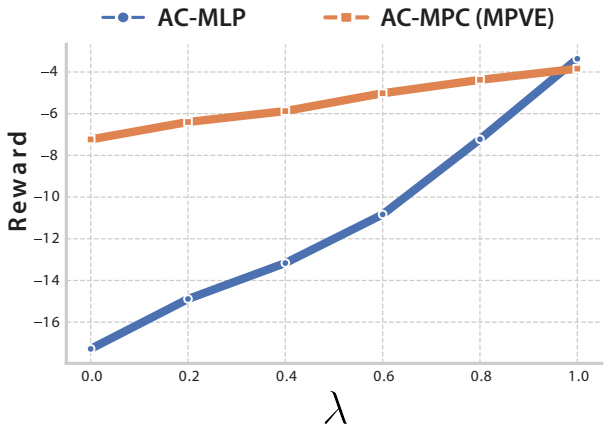
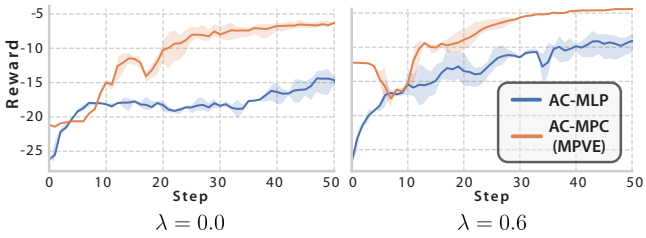


Fig. 7: Reward at 50 steps of training for AC-MLP and AC-MPC (MPVE), depending on the λ used in the TD(λ). The policies are trained with a sample budget of 50 steps and the final reward is recorded, for different values of λ, for both AC-MPC (MPVE) and AC-MLP.



$\lambda = 0.0$ $\lambda = 0.6$

Fig. 8: Reward evolution for the cases of $\lambda = 0.0$ and $\lambda = 0.6$, for AC-MLP and for AC-MPV (MPVE).

In Fig. 8, we showcase the evolution of the episode returns for AC-MLP and AC-MPC-MPVE under two different values of λ and for the same task of drone stabilization. In both settings, our approach substantially outperforms the baseline in terms of asymptotic performance and sample efficiency, highlighting the benefits of model-predictive value expansion.

E. Can we interpret the internal workings of AC-MPC?

This section investigates a potential connection between the critic network's output, $V(\boldsymbol{x})$, and the learned cost matrix $Q(\boldsymbol{x})$. To simplify the analysis, we introduce three key assumptions. First, we assume that the observations s_k are equivalent to the state of the system x_k. Second, the reward function is assumed to be quadratic in the state, expressed as $r(\boldsymbol{x}) = -\boldsymbol{x}^T \boldsymbol{Q} \boldsymbol{x}$. This formulation captures the objective of stabilizing the system around a hover point. And third, we restrict the Model Predictive Control (MPC) horizon length to $T = 2$. Under these simplifications, if we had a linear system with no constraints, the problem becomes equivalent to a Linear Quadratic Regulator (LQR) problem. In LQR, the value function is the solution to the Riccati equation, which is inherently quadratic with respect to the state.

Given the focus on drone dynamics and a stabilization task, we expect the system to primarily operate near hover states. This implies that the learned value function should also be approximately quadratic. The central question we aim to address here is whether a relationship exists between the Hessian (the matrix containing all second-order partial derivatives) of this value function and the learned cost matrix, Q. The general infinite horizon problem for an MPC can be written as:

$$\begin{aligned}
\pi^*(x_0) &= \underset{u}{\operatorname{argmin}} \sum_{k=0}^{\infty} l(x_k, u_k) \\
&= \underset{u}{\operatorname{argmin}} \sum_{k=0}^{N-1} l(x_k, u_k) + \sum_{k=N}^{\infty} l(x_k, u_k) \\
&= \underset{u}{\operatorname{argmin}} \sum_{k=0}^{N-1} l(x_k, u_k) - V^*(x_N) \\
&= \underset{u}{\operatorname{argmin}} \sum_{k=0}^{N-1} l(\tau_k) - V^*(f(\tau_{N-1})),
\end{aligned}$$

where $\tau_k = \begin{bmatrix} x_k, & u_k \end{bmatrix}$ is a state-action pair at time k and the value function term is negative because we have defined it in the context of RL, where it is maximized.

On the other hand, the quadratized problem that we solve in the differentiable MPC block has the following form:

$$\sum_{k=0}^{N} \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T Q_k \begin{bmatrix} x_k \\ u_k \end{bmatrix} + p_k \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \tag{11}$$

This motivates the investigation of a potential relationship between the value function $V(x)$ learned by the critic and the quadratic (Q_N) and linear (p_N) terms learned by the actor within the differentiable MPC framework. In particular, we aim to identify a connection between the first and second derivatives of the value function and p_N and Q_N, respectively. To achieve this, we can introduce the second-order Taylor Series approximation of the value function around the state-action pair τ_{N-1}:

$$\begin{aligned}
V(f(\tau_{N-1}))\Big|_{\tau_{N-1}=\boldsymbol{d}} &\approx V(f(\boldsymbol{d})) \\
&\quad + \tau_{N-1}^T \frac{\partial^2 V(f(\tau_{N-1}=\boldsymbol{d}))}{\partial \tau_{N-1}^2} \tau_{N-1} \\
&\quad + \frac{\partial V(f(\tau_{N-1}=\boldsymbol{d}))}{\partial \tau_{N-1}} \tau_{N-1} \\
&= V(f(\boldsymbol{d})) + \tau_{N-1}^T H_V \tau_{N-1} + \Delta_V \tau_{N-1},
\end{aligned}$$

where $f(\cdot)$ represents the system dynamics function, $\boldsymbol{d}$ denotes a specific state-action pair, H_V is the Hessian of the value function evaluated at $\boldsymbol{d}$, Δ_V represents the first-order term of the Taylor expansion. Because we need to compute the gradient and the Hessian of the value function composed with the dynamics, here are the expressions for Δ_V and H_V.

$$\begin{aligned}
\Delta_V &= \frac{\partial}{\partial \tau_{N-1}} (V(f(\tau_{N-1}))) = \frac{\partial}{\partial \tau_{N-1}} (V(x_N)) \\
&= \frac{\partial V(x_N)}{\partial x_N} \frac{\partial x_N}{\partial \tau_{N-1}} = \frac{\partial V(x_N)}{\partial x_N} \frac{\partial f(\tau_{N-1})}{\partial \tau_{N-1}} \\
&= \frac{\partial V(x_N)}{\partial x_N} \begin{bmatrix} A_{N-1}, B_{N-1} \end{bmatrix},
\end{aligned} \tag{12}$$

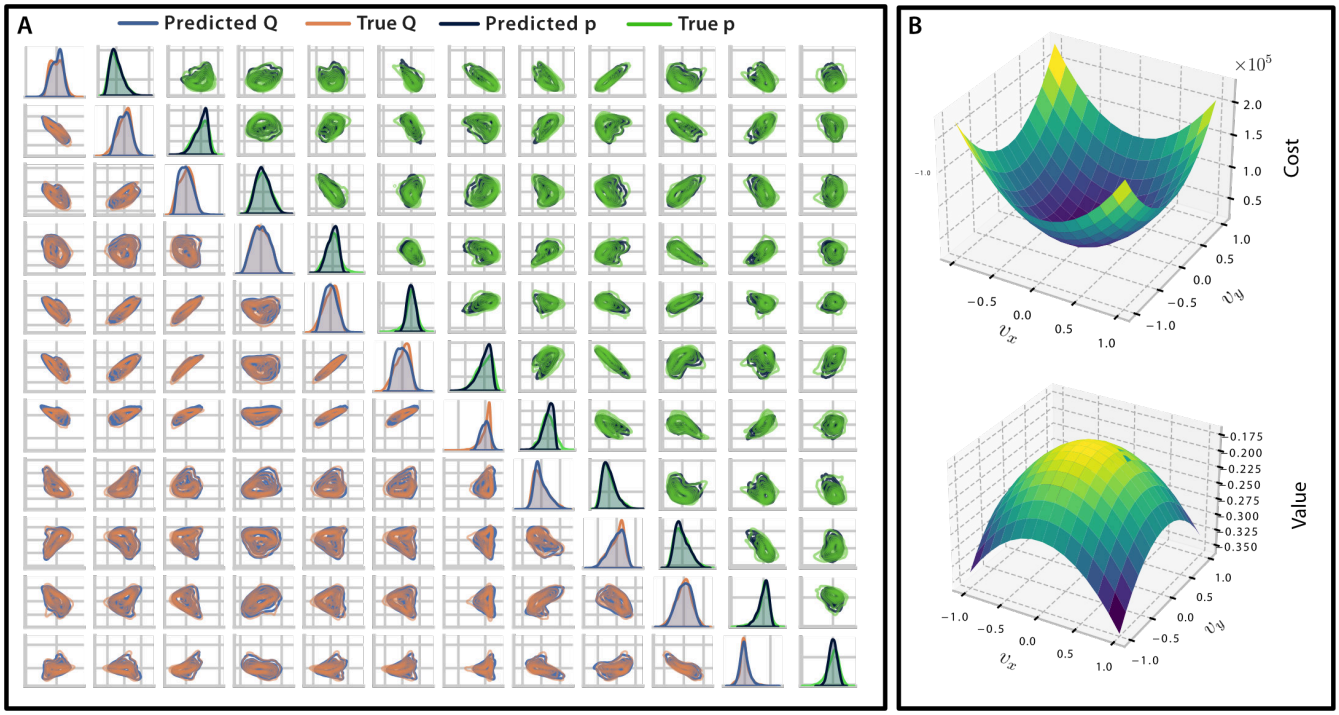


Fig. 9: **Empirical probing of the correlation between the Hessian and the gradient of the learned critic $V(x)$ and the learned differentiable MPC cost terms Q_N and p_N. (A)**: We train a linear classifier to infer Q_N and p_N from the Hessian of $V(x)$ and show the prediction from different 2D projections of Q_N and p_N. We show how the ground truth can be predicted with high accuracy by only using a small linear classifier, which indicates that the data learned by the critic and Q_N and p_N are highly correlated. **(B)**: Top: 3D plot of $\tau_N^T Q_N \tau_N + \tau_N^T p_N$ for the state dimensions of v_x and v_y after training. Bottom: 3D plot of $V(x)$ for the same dimensions. Apart from the change in sign and in scale, the similarity between both shapes indicates a high correlation between the value function and the learned cost parameters.

where $\left[A_{N-1}, B_{N-1}\right]$ are the matrices associated to the linearized dynamics at timestep $N-1$, and $x_N = f(\tau_{N-1})$, $\frac{\partial V(x_N)}{\partial x_N}$ is the gradient of the value function with respect to its inputs. For H_V:

$$
\begin{aligned}
H_V &= \frac{\partial}{\partial \tau_{N-1}}(\Delta_V) \\
&= \frac{\partial}{\partial \tau_{N-1}}\left(\frac{\partial V(x_N)}{\partial x_N}\right)\frac{\partial f(\tau_{N-1})}{\partial \tau_{N-1}} \\
&+ \frac{\partial V(x_N)}{\partial x_N}\cancelto{0}{\frac{\partial^2 f(\tau_{N-1})}{\partial \tau_{N-1}^2}} \\
&= \frac{\partial x_N^T}{\partial \tau_{N-1}}\frac{\partial^2 V(x_N)}{\partial x_N^2}\frac{\partial x_N}{\partial \tau_{N-1}} \\
&= \frac{\partial f(\tau_{N-1})^T}{\partial \tau_{N-1}}\frac{\partial^2 V(x_N)}{\partial x_N^2}\frac{\partial f(\tau_{N-1})}{\partial \tau_{N-1}} \\
&= \begin{bmatrix} A_{N-1} \\ B_{N-1} \end{bmatrix}\frac{\partial^2 V(x_N)}{\partial x_N^2}\left[A_{N-1}, B_{N-1}\right], \qquad (13)
\end{aligned}
$$

where we have assumed that the second derivative of the dynamics is zero, and $\frac{\partial^2 V(x_N)}{\partial x_N^2}$ is the Hessian of the value function with respect to its inputs.

In order to empirically show the relationship between the Hessian of the value function and what is learned in Q_N and p_N, we perform a similarity study using linear probes. Particularly, we first train an AC-MPC agent to perform the stabilization at hover task, introduced in Section V-D. We choose this task here because, since the reward function is quadratic, it results in an optimization landscape that is easier to interpret. Since we need the derivative of the value function with respect to the full state, the critic needs to be trained with full state information. After training, we collect a dataset of uniformly distributed $2^{19} = 524288$ datapoints, where every datapoint corresponds to a sample of Q_N, p_N, H_V and Δ_V, as per Eq. (13) and (12). Then, with this dataset, we train a single linear layer (without non-linearity) to predict the elements of the learned Q_N and p_N matrices from the elements of the H_V and p_V, respectively. In the hypothesized case where the information contained in the learned cost is related to the information encoded by the gradient and Hessian of the value function, we expect the regression to have a high accuracy.

In Fig. 9A, we show the true and predicted values of Q_N and p_N. In order to show that the data fully correlates in all dimensions, the data is presented in a lower triangular matrix for Q and in an upper triangular matrix for p. Every row and column of this matrix is composed of the elements of the state space x, therefore showing the correlation from different 1-dimensional cuts – for the diagonal elements – and 2-dimensional cuts – for the rest. One can therefore see a high correlation between the predicted and the true

values, therefore confirming a strong relationship between the value function and the learned cost parameters. An extra experiment was conducted where we have tried to infer the same Q and p values, but from a random dataset, as a validation that there is no overfitting behavior happening. Indeed, in this case the linear classifier was not able to predict the true labels at all.

In conclusion, this section directly investigates and reveals a relationship within the internal workings of AC-MPC. By demonstrating a strong correlation between the Hessian and gradient of the learned value function and the learned cost parameters Q_N and p_N, we have shown that the critic effectively learns information related to the curvature of the cost landscape used by the MPC module. This provides valuable insight into how the critic informs the MPC controller and suggests that the learned value function encodes information about the system's local dynamics and cost structure.

F. How sensitive is our approach to exploration hyperparameters?

Since our approach incorporates a model of the dynamics within the optimization problem, it has access to prior information before the training phase, and hence leverages prior knowledge of the dynamics in its architecture. This raises questions about the extent of exploration needed to discover an optimal policy and how the exploration parameter can be minimized to achieve optimal performance. To explore this, we study the effect of the initial policy standard deviation hyperparameter on the performance of both AC-MLP and AC-MPC. This hyperparameter influences the exploration behavior of the RL algorithms. In Fig. 10, we present reward plots for the training on the Vertical track under different values of this hyperparameter. The results show that AC-MLP requires a higher degree of exploration to identify the optimal solution. This indicates that AC-MPC effectively utilizes the prior dynamics knowledge, resulting in improved performance with reduced exploration. More interestingly, Fig. 10 shows that AC-MPC is less sensitive to the choice of this hyperparameter and yields high performance despite very low exploration values. On the other hand, AC-MLP fails to reach high-reward regions when trained with low values and generally underperforms in comparison to the proposed method.

G. How does our method perform when deployed in the real-world?

We test our approach in the real world with a high-performance racing drone. In order to test simulation to real world transfer, we deploy the policy with two different race tracks: Circle track and SplitS track. We use the Agilicious control stack [122] for the deployment. The main physical parameters and components of this platform are based on [14], under the name *4s drone*.

Additionally, we perform a lap time comparison in the SplitS track. We train both AC-MPC and AC-MLP in the task of drone racing and execute different runs in two different simulation fidelities: nominal dynamics and realistic

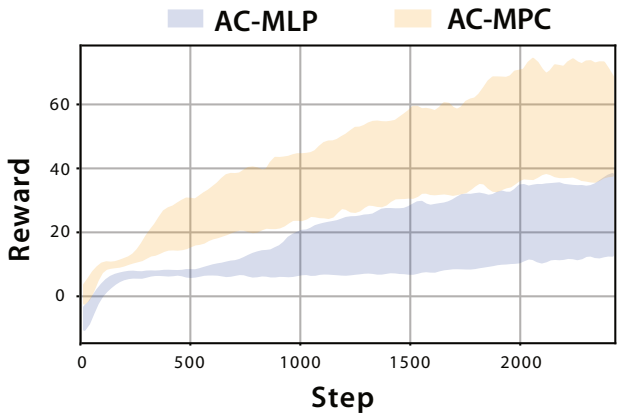


Fig. 10: Comparison of reward values during training for different exploration parameters [0.22, 0.3, 0.44, 0.6] for both AC-MLP and AC-MPC, for the Vertical track.

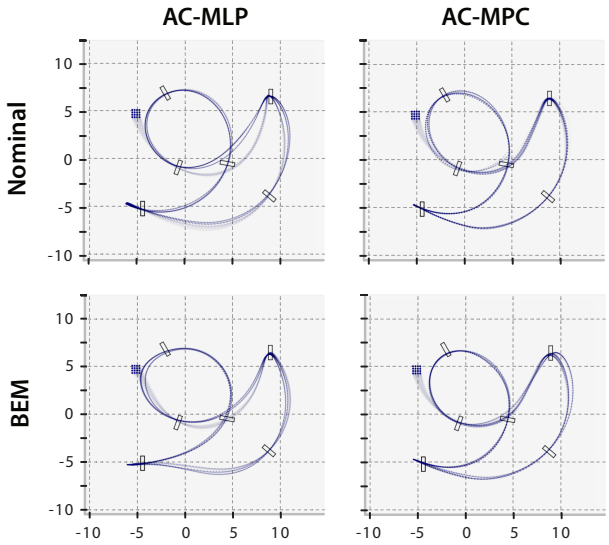
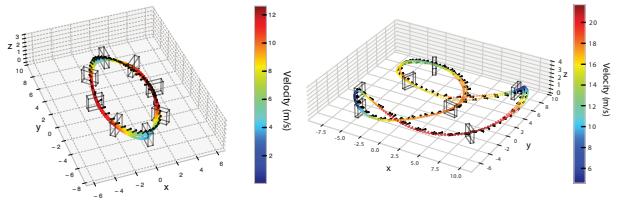


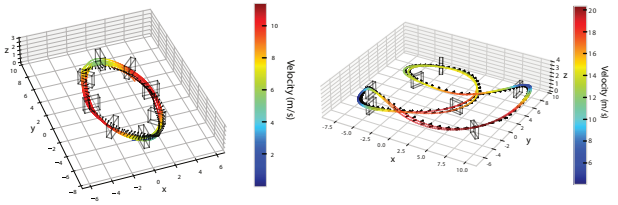
Fig. 11: Top view of the performance of AC-MLP and AC-MPC policies deployed in both nominal and a realistic simulator for the SplitS track. The policies are tested for 64 different initial positions, distributed in a cube of $0.5m$ of side length.

simulator. This realistic simulator, called NeuroBEM, and based in blade element momentum theory, comes from [120]. In Fig. 11 we show a top view of the deployment of both AC-MLP and AC-MPC in both of the aforementioned simulators. The policies are tested for 64 different initial positions, contained in a cube of 0.5 m of side length. Additionally, Fig. 12 illustrates trajectories flown in two different tracks (Circle track and SplitS track) in simulation and the real world. Not only can our policy transfer to the real world without any fine-tuning, but it can do so while at the limit of handling, achieving speeds up to 21 m/s.

In Table II, we show the results of this comparison in terms of lap time and success rate, including both simulation modalities and real-world results. As observed in the lap time results, both AC-MPC and AC-MLP exhibit similar performance in terms of lap time, indicating that they are largely on par in this aspect. This similarity is expected,



(a) Simulation results



(b) Real-world results

Fig. 12: AC-MPC trained for the task of agile flight in complex environments. On the left is the Circle track, and on the right is the SplitS track for both the real world and simulation. These figures show how our approach is able to be deployed in the real world and how they transfer zero shot from simulation to reality. The plots show the flown trajectories by our quadrotor platform, recorded by a motion capture system.

TABLE II: Performance comparison between AC-MLP and AC-MPC in the SplitS track

	AC-MLP		AC-MPC (N=2)	
Drone Model	Lap Time [s]	Success Rate [%]	Lap Time [s]	Success Rate [%]
Nominal	5.09 ± 0.008	100.0	5.13 ± 0.008	100.0
Realistic	5.179 ± 0.01	100.0	5.24 ± 0.01	100.0
Real World	5.39 ± 0.08	85.7	5.4 ± 0.082	87.5

given that their asymptotic performance and final reward values are quite close, as illustrated in Figure 4. However, it is important to note that our approach not only achieves on-par performance but also demonstrates additional robustness to out-of-distribution scenarios (see Fig. 5, in Horizontal and Vertical tracks), and changes in the system dynamics (see Fig. 6, in SplitS track). This indicates that our method maintains its effectiveness under a wider range of conditions and uncertainties, while still matching the performance of the neural network-based architecture.

H. How do training and inference times compare to standard actor-critic architectures?

In Table III, we show the training times (SplitS track) and the forward pass times for AC-MLP and for the proposed AC-MPC for different horizon lengths. As can be seen, while AC-MPC offers performance advantages, it incurs a higher computational cost. Training time is increased by a factor of approximately 30 (for a horizon length of N=2) due to the batched optimization required for each forward and backward pass. This computational overhead also affects online performance, resulting in a slower forward pass. Although the current Python implementation[1] of the differentiable MPC algorithm [26] achieves a sufficient deployment frequency of 50Hz for our system, optimized C/C++ implementations would significantly reduce training time and enable higher frequency deployments. Note, however, that this computational overhead did not affect closed-loop performance, as the 13.5ms execution time remained well within our 20ms control time step budget necessary to deploy the system successfully in the real world.

[1]https://locuslab.github.io/mpc.pytorch/

TABLE III: Solve times and inference times for different variations.

	Training time	Inference time
AC-MLP	21m	0.5 ± 0.037 ms
AC-MPC (N=2)	11h:30m	13.5 ± 1.1 ms
AC-MPC (N=5)	22h:6m	37.5 ± 14.5 ms
AC-MPC (N=10)	39h:36m	69.9 ± 22 ms
AC-MPC (N=50)	-	210.32 ± 22.4 ms

VI. Discussion and Conclusion

This work presented a new learning-based control framework that combines the advantage of differentiable model predictive control with actor-critic training. Furthermore, we showed that AC-MPC can leverage the prior knowledge embedded in the system dynamics to achieve better training performance (sample efficiency, robustness to hyperparameters) than the AC-MLP baseline, and to better cope with out-of-distribution scenarios, and with variations in the nominal dynamics, without any further re-training. Through empirical analysis, the interpretable nature of the differentiable MPC embedded in the actor-critic architecture allows us to conduct and empirical analysis that revealed a relationship between the learned value function and the learned cost functions. This provides a deeper understanding of the interplay between the reinforcement learning (RL) and MPC, offering valuable insights for future research. Additionally, our approach achieved zero-shot sim-to-real transfer, demonstrated by successfully controlling a quadrotor at velocities of up to 21 m/s in the physical world, being on par in performance with the AC-MLP baseline. In general, we show that our method can tackle challenging control tasks and achieves robust control performance for agile flight.

However, there are some limitations to be mentioned and to be improved in the future. First, an analytic model of the system is required for the differentiable MPC block, which limits our approach to mainly systems where the dynamics are known beforehand. Furthermore, training AC-MPC takes significantly longer than AC-MLP (see Table III), due to the fact that an optimization problem needs to be solved for both the forward and the backward pass through the actor network. In fact, there are open-source libraries [99], [100] that are recently evolving and implementing more efficient versions of differentiable MPC. Showcasing the approach in different tasks with different robots is another future direction.

We believe that our method represents an important step in the direction of generalizability and robustness in RL. The proposed method demonstrates that modular solutions, which

combine the advantages of learning-centric and model-based approaches, are becoming increasingly promising. Our approach potentially paves the way for the development of more robust RL-based systems, contributing positively towards the broader goal of advancing AI for real-world robotics applications.

ACKNOWLEDGMENTS

We would like to thank Brandon Amos, for sharing his insights regarding the differentiable MPC code, and Sebastien Gros, for taking the time to proofread our manuscript. We would also like to thank Jiaxu Xing, Leonard Bauersfeld and Alex Barden for the insightful discussions and support with the hardware setup.

REFERENCES

[1] R. P. N. Rao and D. H. Ballard, "Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects," *Nature Neuroscience*, vol. 2, no. 1, pp. 79–87, 1999.

[2] K. Friston, "The free-energy principle: a unified brain theory?" *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010.

[3] Y. LeCun, "A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27," *Open Review*, vol. 62, 2022.

[4] T. Tzanetos, M. Aung, J. Balaram, H. F. Grip, J. T. Karras, T. K. Canham, G. Kubiak, J. Anderson, G. Merewether, M. Starch, M. Pauken, S. Cappucci, M. Chase, M. Golombek, O. Toupet, M. C. Smart, S. Dawson, E. B. Ramirez, J. Lam, R. Stern, N. Chahat, J. Ravich, R. Hogg, B. Pipenberg, M. Keennon, and K. H. Williford, "Ingenuity mars helicopter: From technology demonstration to extraterrestrial scout," in *2022 IEEE Aerospace Conference (AERO)*. IEEE, 2022, pp. 01–19.

[5] E. Arthur Jr and J.-C. Ho, *Applied optimal control: optimization, estimation, and control*. Hemisphere, 1975.

[6] M. Ellis, J. Liu, and P. D. Christofides, *Economic Model Predictive Control: Theory, Formulations and Chemical Process Applications*. Springer, 2016.

[7] P.-B. Wieber, R. Tedrake, and S. Kuindersma, "Modeling and control of legged robots," in *Springer handbook of robotics*. Springer, 2016, pp. 1203–1234.

[8] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, "Alphapilot: Autonomous drone racing," *Robotics: Science and Systems (RSS)*, 2020.

[9] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Science Robotics*, vol. 6, no. 56, 2021.

[10] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, 2022.

[11] A. Romero, R. Penicka, and D. Scaramuzza, "Time-optimal online replanning for agile quadrotor flight," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7730–7737, 2022.

[12] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 709–715.

[13] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[14] Y. Song, A. Romero, M. Mueller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, p. adg1462, 2023.

[15] N. Roy, I. Posner, T. D. Barfoot, P. Beaudoin, Y. Bengio, J. Bohg, O. Brock, I. Depatie, D. Fox, D. E. Koditschek, T. Lozano-Perez, V. K. Mansinghka, C. J. Pal, B. A. Richards, D. Sadigh, S. Schaal, G. S. Sukhatme, D. Thérien, M. Toussaint, and M. van de Panne, "From machine learning to robotics: Challenges and opportunities for embodied intelligence," *ArXiv*, vol. abs/2110.15245, 2021.

[16] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion planning and control for mobile robot navigation using machine learning: a survey," *Autonomous Robots*, vol. 46, no. 5, pp. 569–597, 2022.

[17] D. Silver, A. Huang, C. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[18] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: http://dx.doi.org/10.1038/nature14236

[19] O. Vinyals, I. Babuschkin, W. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[20] P. R. Wurman, S. Barrett, K. Kawamoto, J. MacGlashan, K. Subramanian, T. J. Walsh, R. Capobianco, A. Devlic, F. Eckert, F. Fuchs *et al.*, "Outracing champion gran turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.

[21] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, Aug 2023.

[22] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

[23] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-based model predictive control: Toward safe learning in control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, pp. 269–296, 2020.

[24] K. P. Wabersich, L. Hewing, A. Carron, and M. N. Zeilinger, "Probabilistic model predictive safety certification for learning-based control," *IEEE Transactions on Automatic Control*, vol. 67, no. 1, pp. 176–188, 2021.

[25] A. Romero, Y. Song, and D. Scaramuzza, "Actor-critic model predictive control," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 777–14 784.

[26] B. Amos, I. Jimenez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable mpc for end-to-end planning and control," *Advances in neural information processing systems*, vol. 31, 2018.

[27] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[28] Y. Wang and S. Boyd, "Fast model predictive control using online optimization," *IEEE Transactions on control systems technology*, vol. 18, no. 2, pp. 267–278, 2009.

[29] R. González, M. Fiacchini, J. L. Guzmán, T. Álamo, and F. Rodríguez, "Robust tube-based predictive control for mobile robots in off-road conditions," *Robotics and Autonomous Systems*, vol. 59, no. 10, pp. 711–726, 2011.

[30] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1: 43 scale rc cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.

[31] F. Farshidian, E. Jelavic, A. Satapathy, M. Giftthaler, and J. Buchli, "Real-time motion planning of legged robots: A model predictive control approach," in *2017 IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids)*. IEEE, 2017, pp. 577–584.

[32] C. Liu, S. Lee, S. Varnhagen, and H. E. Tseng, "Path planning for autonomous vehicles using model predictive control," in *2017 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2017, pp. 174–179.

[33] J. Xing, G. Cioffi, J. Hidalgo-Carrió, and D. Scaramuzza, "Autonomous power line inspection with drones via perception-aware mpc," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1086–1093.

[34] T. Han, A. Liu, A. Li, A. Spitzer, G. Shi, and B. Boots, "Model predictive control for aggressive driving over uneven terrain," *arXiv preprint arXiv:2311.12284*, 2023.

[35] E. Arcari, M. V. Minniti, A. Scampicchio, A. Carron, F. Farshidian, M. Hutter, and M. N. Zeilinger, "Bayesian multi-task learning mpc for robotic mobile manipulation," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3222–3229, 2023.

[36] A. Saviolo, J. Frey, A. Rathod, M. Diehl, and G. Loianno, "Active learning of discrete-time dynamics for uncertainty-aware model predictive control," *IEEE Transactions on Robotics*, 2023.

[37] G. Li and G. Loianno, "Nonlinear model predictive control for cooperative transportation and manipulation of cable suspended payloads with multiple quadrotors," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 5034–5041.

[38] L. P. Fröhlich, C. Küttel, E. Arcari, L. Hewing, M. N. Zeilinger, and A. Carron, "Contextual tuning of model predictive control for autonomous racing," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 10 555–10 562.

[39] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, 2022.

[40] M. Krinner, A. Romero, L. Bauersfeld, M. Zeilinger, A. Carron, and D. Scaramuzza, "Mpcc++: Model predictive contouring control for time-optimal flight with safety constraints," *Proceedings of Robotics: Science and Systems, Delft, Netherlands*, 2024.

[41] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.

[42] A. Tagliabue and J. P. How, "Efficient deep learning of robust policies from mpc using imitation and tube-guided data augmentation," *IEEE Transactions on Robotics*, 2024.

[43] ——, "Tube-nerf: Efficient imitation learning of visuomotor policies from mpc via tube-guided data augmentation and nerfs," *IEEE Robotics and Automation Letters*, 2024.

[44] F. Djeumou, T. J. Lew, N. DING, M. Thompson, M. Suminaka, M. Greiff, and J. Subosits, "One model to drift them all: Physics-informed conditional diffusion model for driving at the limits," in *8th Annual Conference on Robot Learning*.

[45] L. Jaillet, J. Cortés, and T. Siméon, "Sampling-based path planning on configuration-space costmaps," *IEEE Transactions on Robotics*, vol. 26, no. 4, pp. 635–646, 2010.

[46] K. Zheng, *ROS Navigation Tuning Guide*. Cham: Springer International Publishing, 2021, pp. 197–226.

[47] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, "Mit cheetah 3: Design and control of a robust, dynamic quadruped robot," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 2245–2252.

[48] M. Neunert, M. Stäuble, M. Giftthaler, C. D. Bellicoso, J. Carius, C. Gehring, M. Hutter, and J. Buchli, "Whole-body nonlinear model predictive control through contacts for quadrupeds," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1458–1465, 2018.

[49] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8.

[50] M. Neunert, C. De Crousaz, F. Furrer, M. Kamel, F. Farshidian, R. Siegwart, and J. Buchli, "Fast nonlinear model predictive control for unified trajectory optimization and tracking," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 1398–1404.

[51] M. Bjelonic, R. Grandia, M. Geilinger, O. Harley, V. S. Medeiros, V. Pajovic, E. Jelavic, S. Coros, and M. Hutter, "Offline motion libraries and online mpc for advanced mobility skills," *The International Journal of Robotics Research*, p. 02783649221102473.

[52] S. Kuindersma, R. Deits, M. Fallon, A. Valenzuela, H. Dai, F. Permenter, T. Koolen, P. Marion, and R. Tedrake, "Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot," *Autonomous robots*, vol. 40, no. 3, pp. 429–455, 2016.

[53] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.

[54] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots, "Storm: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Conference on Robot Learning*. PMLR, 2022, pp. 750–759.

[55] M. Minarik, R. Penicka, V. Vonasek, and M. Saska, "Model predictive path integral control for agile unmanned aerial vehicles," *arXiv preprint arXiv:2407.09812*, 2024.

[56] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing," *arXiv preprint arXiv:2409.15610*, 2024.

[57] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray *et al.*, "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[58] J. Degrave, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de las Casas, C. Donner, L. Fritz, C. Galperti, A. Huber, J. Keeling, M. Tsimpoukelli, J. Kay, A. Merle, J.-M. Moret, S. Noury, F. Pesamosca, D. Pfau, O. Sauter, C. Sommariva, S. Coda, B. Duval, A. Fasoli, P. Kohli, K. Kavukcuoglu, D. Hassabis, and M. Riedmiller, "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, pp. 414–419, 2022.

[59] D. Mankowitz, A. Michi, A. Zhernov *et al.*, "Faster sorting algorithms discovered using deep reinforcement learning," *Nature*, vol. 618, pp. 257–263, 2023.

[60] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[61] A. Gharib, D. Stenger, R. Ritschel, and R. Voßwinkel, "Multi-objective optimization of a path-following mpc for vehicle guidance: A bayesian optimization approach," in *2021 European Control Conference (ECC)*, 2021, pp. 2197–2204.

[62] A. Romero, S. Govil, G. Yilmaz, Y. Song, and D. Scaramuzza, "Weighted maximum likelihood for controller tuning," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 1334–1341.

[63] Y. Song and D. Scaramuzza, "Policy search for model predictive control with application to agile drone flight," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2114–2130, 2022.

[64] A. Romero, P. N. Beuchat, Y. R. Stürz, R. S. Smith, and J. Lygeros, "Nonlinear control of quadcopters via approximate dynamic programming," in *2019 18th European Control Conference (ECC)*. IEEE, 2019, pp. 3752–3759.

[65] B. Tearle, K. P. Wabersich, A. Carron, and M. N. Zeilinger, "A predictive safety filter for learning-based racing control," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7635–7642, 2021.

[66] G. Grandesso, E. Alboni, G. P. R. Papini, P. M. Wensing, and A. Del Prete, "Cacto: Continuous actor-critic with trajectory optimization—towards global optimality," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3318–3325, 2023.

[67] D. Hoeller, F. Farshidian, and M. Hutter, "Deep value model predictive control," in *Conference on Robot Learning*. PMLR, 2020, pp. 990–1004.

[68] R. Reiter, A. Ghezzi, K. Baumgärtner, J. Hoffmann, R. D. McAllister, and M. Diehl, "Ac4mpc: Actor-critic reinforcement learning for nonlinear model predictive control," *arXiv preprint arXiv:2406.03995*, 2024.

[69] J. Sacks and B. Boots, "Learning to optimize in model predictive control," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 10 549–10 556.

[70] J. Sacks, R. Rana, K. Huang, A. Spitzer, G. Shi, and B. Boots, "Deep model predictive optimization," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 16 945–16 953.

[71] B. Zarrouki, M. Spanakakis, and J. Betz, "A safe reinforcement learning driven weights-varying model predictive control for autonomous vehicle motion control," *arXiv preprint arXiv:2402.02624*, 2024.

[72] B. Zarrouki, V. Klös, N. Heppner, S. Schwan, R. Ritschel, and R. Voßwinkel, "Weights-varying mpc for autonomous vehicle guidance: a deep reinforcement learning approach," in *2021 European Control Conference (ECC)*. IEEE, 2021, pp. 119–125.

[73] F. Jenelten, J. He, F. Farshidian, and M. Hutter, "Dtc: Deep tracking control–a unifying approach to model-based planning and reinforcement-learning for versatile and robust locomotion," *arXiv preprint arXiv:2309.15462*, 2023.

[74] R. Reiter, J. Hoffmann, J. Boedecker, and M. Diehl, "A hierarchical approach for strategic motion planning in autonomous racing," in *2023 European Control Conference (ECC)*. IEEE, 2023, pp. 1–8.

[75] K. Seel, A. B. Kordabad, S. Gros, and J. T. Gravdahl, "Convex neural network-based cost modifications for learning model predictive control," *IEEE Open Journal of Control Systems*, vol. 1, pp. 366–379, 2022.

[76] A. Saviolo, G. Li, and G. Loianno, "Physics-inspired temporal learning of quadrotor dynamics for accurate model predictive trajectory tracking," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 256–10 263, 2022.

[77] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, "Data-driven mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.

[78] I. Lenz, R. A. Knepper, and A. Saxena, "Deepmpc: Learning deep latent features for model predictive control." in *Robotics: Science and Systems*, vol. 10. Rome, Italy, 2015.

[79] M. Watter, J. Springenberg, J. Boedecker, and M. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," *Advances in neural information processing systems*, vol. 28, 2015.

[80] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine, "Visual foresight: Model-based deep reinforcement learning for vision-based robotic control," *arXiv preprint arXiv:1812.00568*, 2018.

[81] N. A. Hansen, H. Su, and X. Wang, "Temporal difference learning for model predictive control," in *International Conference on Machine Learning*. PMLR, 2022, pp. 8387–8406.

[82] M. Pereira, D. D. Fan, G. N. An, and E. Theodorou, "Mpc-inspired neural network policies for sequential decision making," *arXiv preprint arXiv:1802.05803*, 2018.

[83] S. Adhau, S. Gros, and S. Skogestad, "Reinforcement learning based mpc with neural dynamical models," *European Journal of Control*, p. 101048, 2024.

[84] E. Bøhn, S. Gros, S. Moe, and T. A. Johansen, "Optimization of the model predictive control meta-parameters through reinforcement learning," *Engineering Applications of Artificial Intelligence*, vol. 123, p. 106211, 2023.

[85] H. N. Esfahani, A. B. Kordabad, W. Cai, and S. Gros, "Learning-based state estimation and control using mhe and mpc schemes with imperfect models," *European Journal of Control*, vol. 73, p. 100880, 2023.

[86] A. S. Anand, D. Reinhardt, S. Sawant, J. T. Gravdahl, and S. Gros, "A painless deterministic policy gradient method for learning-based mpc," in *2023 European Control Conference (ECC)*. IEEE, 2023, pp. 1–7.

[87] W. Cai, S. Sawant, D. Reinhardt, S. Rastegarpour, and S. Gros, "A learning-based model predictive control strategy for home energy management systems," *IEEE Access*, 2023.

[88] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*.

[89] E. Aljalbout, N. Sotirakis, P. van der Smagt, M. Karl, and N. Chen, "Limt: Language-informed multi-task visual world models," *arXiv preprint arXiv:2407.13466*, 2024.

[90] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 7559–7566.

[91] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," *Advances in neural information processing systems*, vol. 31, 2018.

[92] N. Hansen, H. Su, and X. Wang, "Td-mpc2: Scalable, robust world models for continuous control," 2024.

[93] A. Nagabandi, K. Konolige, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *Conference on Robot Learning*. PMLR, 2020, pp. 1101–1112.

[94] H. Sikchi, W. Zhou, and D. Held, "Learning off-policy with online planning," in *Conference on Robot Learning*. PMLR, 2022, pp. 1622–1633.

[95] S. Cheng, L. Song, M. Kim, S. Wang, and N. Hovakimyan, "Difftune$^+$: Hyperparameter-free auto-tuning using auto-differentiation," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, N. Matni, M. Morari, and G. J. Pappas, Eds., vol. 211. PMLR, 15–16 Jun 2023, pp. 170–183.

[96] F. Yang, C. Wang, C. Cadena, and M. Hutter, "iplanner: Imperative path planning," *Robotics: Science and Systems Conference (RSS)*, 2023.

[97] P. Karkus, B. Ivanovic, S. Mannor, and M. Pavone, "Diffstack: A differentiable and modular control stack for autonomous vehicles," in *Conference on Robot Learning*. PMLR, 2023, pp. 2170–2180.

[98] B. Wang, Z. Ma, S. Lai, and L. Zhao, "Neural moving horizon estimation for robust flight control," *IEEE Transactions on Robotics*, 2023.

[99] L. Pineda, T. Fan, M. Monge, S. Venkataraman, P. Sodhi, R. T. Chen, J. Ortiz, D. DeTone, A. Wang, S. Anderson, J. Dong, B. Amos, and M. Mukadam, "Theseus: A Library for Differentiable Nonlinear Optimization," *Advances in Neural Information Processing Systems*, 2022.

[100] C. Wang, D. Gao, K. Xu, J. Geng, Y. Hu, Y. Qiu, B. Li, F. Yang, B. Moon, A. Pandey, Aryan, J. Xu, T. Wu, H. He, D. Huang, Z. Ren, S. Zhao, T. Fu, P. Reddy, X. Lin, W. Wang, J. Shi, R. Talak, K. Cao, Y. Du, H. Wang, H. Yu, S. Wang, S. Chen, A. Kashyap, R. Bandaru, K. Dantu, J. Wu, L. Xie, L. Carlone, M. Hutter, and S. Scherer, "PyPose: A library for robot learning with physics-based optimization," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[101] S. East, M. Gallieri, J. Masci, J. Koutnik, and M. Cannon, "Infinite-horizon differentiable model predictive control," in *International Conference on Learning Representations*, 2019.

[102] X. Xiao, T. Zhang, K. M. Choromanski, T.-W. E. Lee, A. Francis, J. Varley, S. Tu, S. Singh, P. Xu, F. Xia, S. M. Persson, D. Kalashnikov, L. Takayama, R. Frostig, J. Tan, C. Parada, and V. Sindhwani, "Learning model predictive controllers with real-time attention for real-world navigation," in *CoRL*, 2022.

[103] H. Nguyen, M. Kamel, K. Alexis, and R. Siegwart, "Model predictive control for micro aerial vehicles: A survey," *2021 European Control Conference (ECC)*, 2021.

[104] M. Bangura and R. Mahony, "Real-time model predictive control for quadrotors," *IFAC World Congress*, 2014.

[105] M. Diehl, H. G. Bock, H. Diedam, and P. B. Wieber, "Fast direct multiple shooting algorithms for optimal robot control," in *Fast motions in biomechanics and robotics*. Springer, 2006.

[106] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing with deep reinforcement learning," *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[107] R. Penicka and D. Scaramuzza, "Minimum-time quadrotor waypoint flight in cluttered environments," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5719–5726, 2022.

[108] M. Ellis, H. Durand, and P. D. Christofides, "A tutorial review of economic model predictive control methods," *Journal of Process Control*, vol. 24, no. 8, pp. 1156–1178, 2014.

[109] J. B. Rawlings, D. Angeli, and C. N. Bates, "Fundamentals of economic model predictive control," in *2012 IEEE 51st IEEE conference on decision and control (CDC)*. IEEE, 2012, pp. 3851–3861.

[110] S. Gros and M. Zanon, "Data-driven economic nmpc using reinforcement learning," *IEEE Transactions on Automatic Control*, vol. 65, no. 2, pp. 636–648, 2019.

[111] R. Verschueren, M. Zanon, R. Quirynen, and M. Diehl, "Time-optimal race car driving using an online exact hessian based nonlinear mpc algorithm," in *2016 European control conference (ECC)*. IEEE, 2016, pp. 141–147.

[112] M. Zanon, "A gauss–newton-like hessian approximation for economic nmpc," *IEEE Transactions on Automatic Control*, vol. 66, no. 9, pp. 4206–4213, 2020.

[113] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM Journal on control and optimization*, vol. 43, no. 5, pp. 1714–1736, 2005.

[114] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear mpc: bridging the gap via the real-time iteration," *International Journal of Control*, vol. 93, no. 1, pp. 62–80, 2020.

[115] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados – a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, 2021.

[116] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Advances in neural information processing systems*, vol. 12, 1999.

[117] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv e-prints*, 2017.

[118] Y. Song and D. Scaramuzza, "Learning high-level policies for model predictive control," in *IEEE/RSJ Int. Conf. Intell. Robot. Syst. (IROS)*, 2020.

[119] V. Feinberg, A. Wan, I. Stoica, M. I. Jordan, J. E. Gonzalez, and S. Levine, "Model-based value estimation for efficient model-free reinforcement learning," *arXiv preprint arXiv:1803.00101*, 2018.

[120] L. Bauersfeld, E. Kaufmann, P. Foehn, S. Sun, and D. Scaramuzza, "Neurobem: Hybrid aerodynamic quadrotor model," *Proceedings of Robotics: Science and Systems XVII*, p. 42, 2021.

[121] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *Conference on Robot Learning*, 2020.

[122] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, vol. 7, no. 67, p. eabl6259, 2022.

[123] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, "A benchmark comparison of learned control policies for agile quadrotor flight," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 10 504–10 510.

[124] E. Aljalbout, F. Frank, M. Karl, and P. van der Smagt, "On the role of the action space in robot manipulation learning and sim-to-real transfer," *IEEE Robotics and Automation Letters*, 2024.

[125] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.